

Nova Bank: Technical Architecture & System Integration Report

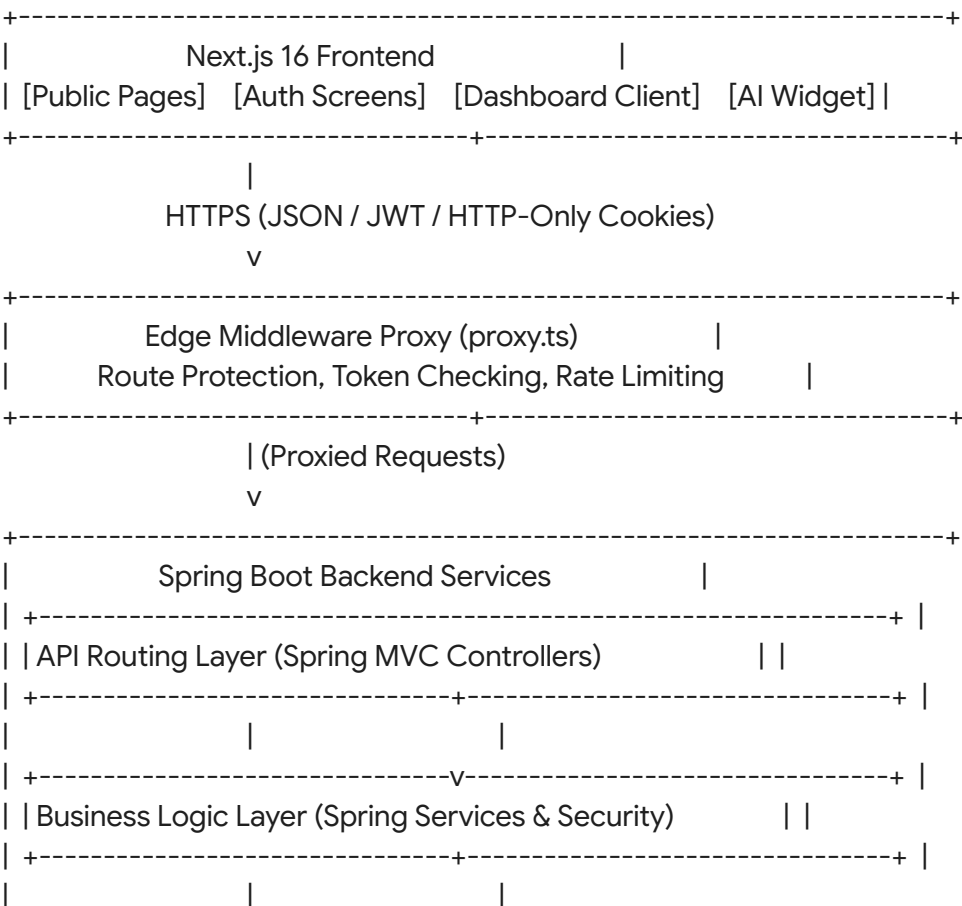
Project Framework: Hack-To-Night 2026 Challenge / Nova Bank

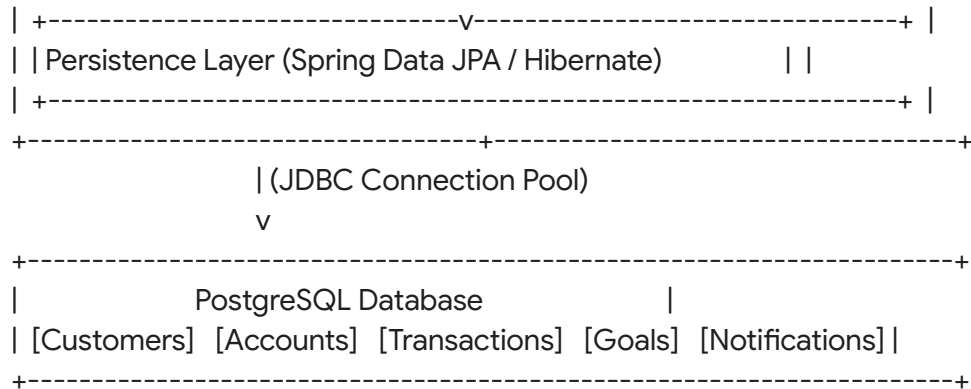
Core Technologies:

- **Frontend UI/UX:** Next.js 16 (App Router), React 19, Tailwind CSS, Biome Linter/Formatter
- **Backend Services:** Java 21, Spring Boot 3.x, Spring Security, Hibernate ORM
- **Database Management:** PostgreSQL 16 (Relational Core, ACID Transactions)
- **Integrations:** Firebase Auth (OIDC), Nodemailer / Spring Mailer, AI Chatbot engine

1. Global System Architecture

Nova Bank uses a fully decoupled client-server architecture. The frontend application, built on Next.js 16, acts as a high-performance presentation and orchestration layer that communicates asynchronously with a resilient Java Spring Boot backend via JSON REST APIs. Data persistence is managed by PostgreSQL, leveraging strict relational constraints, isolation levels, and transactional guarantees to protect financial ledgers.



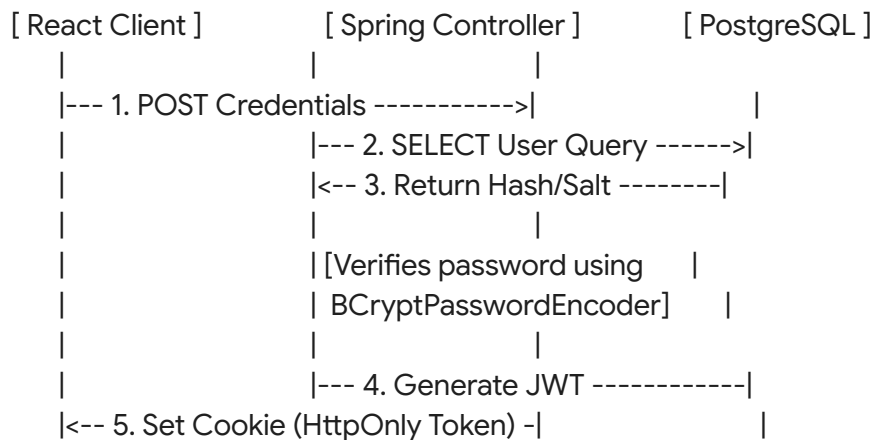


2. Integrated System Flows

This section details the end-to-end logical processing sequences spanning the React 19 client components, Spring MVC endpoint mappings, Java service execution branches, and PostgreSQL transaction commands.

Flow A: Client Authentication & Session Handshake

- **Target Routes:** /login, /sign-up, /reset-password
- **API Interconnects:** POST /api/auth/login, POST /api/auth/signup, GET /api/auth/me, POST /api/auth/firebase



Technical Breakdown:

1. **Frontend Dispatch:** The React 19 signup page uses a client-side component to collect registration inputs, validating schema constraints via client utility models before calling the API.
2. **Backend Authentication:** The Java backend handles the incoming REST request inside an AuthController. Spring Security intercepts unsecured requests, delegating credentials verification to a custom UserDetailsService.

- 3. **Database Security:** User tables are queried securely using parameterized Hibernate ORM mappings to prevent SQL injection vulnerabilities.
- 4. **JWT Generation:** Upon successful authentication, the backend generates an encrypted JWT string, containing appropriate claim metadata. This token is packaged into a secure HttpOnly cookie payload and sent to the client browser to protect active credentials from Cross-Site Scripting (XSS) access.

Flow B: Accounts & Core Dashboard Metrics Hydration

- **Target Routes:** /dashboard, /bank-accounts, /accounts
- **API Interconnects:** GET /api/accounts, POST /api/accounts, PUT /api/accounts, DELETE /api/accounts

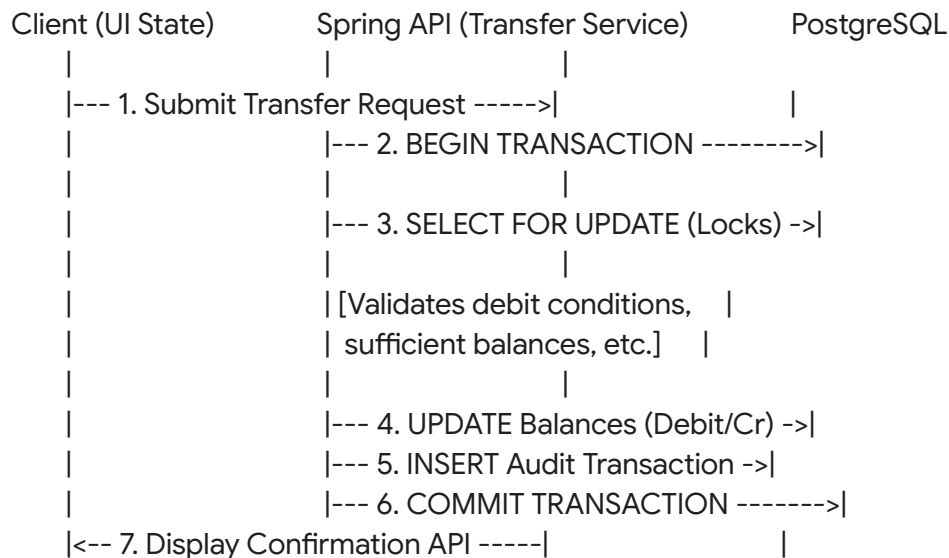
Next.js Page (Client Mount)	Spring Boot Controller (DashboardService)	PostgreSQL (MVCC Read Engine)
--- 1. Fetch Accounts & Summary --->		
	--- 2. SELECT ACCOUNTS -----	
	--- & TRANSACTIONS ----->	
	[Calculates aggregate balances,	
	spending percentages, trends]	
	<--- 3. Return Relational Rows ---	
<-- 4. Dynamic UI Layout Render ----		

Technical Breakdown:

- 1. **Initial Dashboard Mount:** The Next.js /dashboard root template utilizes standard fetch mechanisms (or React Hooks like SWR / React Query) to load active balances and current month expenditures.
- 2. **Java Query Parallelization:** Inside the backend DashboardService, parallel database read operations are executed asynchronously via @Async Spring methods. This isolates transaction scopes and optimizes API response latency:
 - Query 1: Active bank accounts belonging to the authenticated customer ID.
 - Query 2: Rolling aggregation of expenditures grouped by spending categories for the current billing period.
 - Query 3: Recent transaction ledger history (paginated up to 10 entries).
- 3. **PostgreSQL Execution:** Reads run using the standard READ_COMMITTED transaction isolation level, preventing dirty reads while utilizing PostgreSQL's Multi-Version Concurrency Control (MVCC) to ensure queries do not block write transactions on other threads.

Flow C: ACID-Compliant Funds Transfer Execution

- **Target Routes:** /bank-transfer
- **API Interconnects:** POST /api/transfer, POST /api/beneficiaries



Technical Breakdown:

1. **Interactive Pin Lock:** The transfer interface captures user input, recipient details, and secondary transaction PIN confirmations.
2. **Strict Isolation Control:** The Spring Boot backend service exposes the /api/transfer endpoint within a highly isolated transaction boundary to prevent concurrent balance overdrafts:

```
@Transactional(isolation = Isolation.SERIALIZABLE, rollbackFor = Exception.class)
public TransferResponse executeFundsTransfer(TransferRequest request) { ... }
```

3. **Row-Level Mutual Exclusion:** To eliminate race conditions where a customer attempts rapid concurrent transfers, the query engine applies exclusive row-level locks within PostgreSQL:

```
SELECT balance FROM bank_accounts WHERE account_number = ? FOR UPDATE;
```

4. **Debit/Credit Calculations:** Let B_{sender} be the original sender balance, $B_{recipient}$ be the recipient balance, and W be the transfer amount. The system evaluates the following state transition:

$$B_{sender_new} = B_{sender} - W$$

$$B_{recipient_new} = B_{recipient} + W$$

This transition succeeds if and only if $B_{sender} \geq W$ and $W > 0$. If validated, balances are updated and an entry is generated within the relational transactions table. If any processing step fails, the Hibernate transaction triggers an automatic ROLLBACK to restore original balances.

Flow D: Smart Spend Analytics & Monthly Budgeting

- **Target Routes:** /smart-spend
- **API Interconnects:** GET /api/smart-spend, POST /api/smart-spend, PUT /api/smart-spend

Technical Breakdown:

1. **Aggregation Engine:** Next.js mounts interactive charting components. It requests categorized metrics over parameterized historical intervals (e.g., 30, 60, or 90 days).
2. **Java Analytical Mapping:** The Spring Boot service runs analytical queries against the database. Instead of loading thousands of individual transactions into memory, it utilizes optimized database aggregation:

```
SELECT category, SUM(amount) AS total_amount, COUNT(id) AS frequency
FROM transactions
WHERE customer_id = ? AND transaction_date >= ?
GROUP BY category;
```

3. **Budget Progress Metrics:** The system computes progress metrics based on budget configurations. If target threshold constraints C_{limit} and current category spend sums $S_{category}$ are loaded, progress percentages P are calculated as:

$$P = \left(\frac{S_{category}}{C_{limit}} \right) \times 100$$

This structured metadata is returned to the React 19 frontend to render responsive progress bars.

Flow E: On-Demand E-Statements Document Generation

- **Target Routes:** /e-statement
- **API Interconnects:** GET /api/statement

Technical Breakdown:

1. **Filtered Date Range Request:** The client specifies target accounts and query criteria (e.g., start_date and end_date).

2. **Memory-Stream Processing:** The backend uses Spring MVC to construct document files dynamically rather than retrieving static files. It streams database query results directly into a transient memory buffer.
3. **Database Streaming:** For accounts with large transaction volumes, the backend utilizes Java Stream<Transaction> patterns alongside Hibernate's scrollable result sets to stream data directly from PostgreSQL:

```
@Query("SELECT t FROM Transaction t WHERE t.accountNumber = :accNum AND t.date BETWEEN :start AND :end")
Stream<Transaction> streamTransactions(@Param("accNum") String acc, @Param("start") LocalDate s, @Param("end") LocalDate e);
```
4. **Response Assembly:** The records are assembled into a structured CSV or PDF byte array inside application memory and returned to the frontend client with explicit header mappings:
Content-Disposition: attachment; filename="statement.pdf"

Flow F: Savings Goals & Milestone Tracking

- **Target Routes:** /savings-goals
- **API Interconnects:** GET /api/savings-goals, POST /api/savings-goals, PUT /api/savings-goals, DELETE /api/savings-goals

Technical Breakdown:

1. **Component Initialization:** The UI displays active goals, savings achievements, and automatically computed contribution schedules.
2. **Persistent Management:** The backend runs CRUD logic on the savings_goals table, verifying ownership boundaries on every write operation.
3. **Pacing Calculations:** The server calculates the daily savings pace required to reach target milestones. If G represents the total savings goal target, A represents the current saved amount, and $D_{remaining}$ represents the remaining calendar days:

$$\text{Daily Pacing Rate} = \frac{G - A}{D_{remaining}}$$

These calculations are performed in real time when goals are modified, and the resulting pacing schedule is returned directly to the frontend.

Flow G: Settings Modification & Notification Queuing

- **Target Routes:** /profile, /settings, /notifications
- **API Interconnects:** GET /api/profile, PUT /api/profile, GET /api/notifications

Client (UI Interceptor) Java Security/Profile Service PostgreSQL

```

|                                     |
|--- 1. PUT Profile Update ----->|                                     |
|                                     |--- 2. UPDATE Customers ----->|
|                                     |
|                                     | [Asynchronously triggers      |
|                                     | notification event listener] |
|                                     |
|                                     |--- 3. INSERT Notification ----->|
|<--- 4. Emit HTTP 200 Success -----|                                     |

```

Technical Breakdown:

1. **Interactive Configuration:** Users manage local UI parameters, change personal details, or toggle email alerts via /settings.
2. **Audit Event Dispatch:** When critical security or configuration changes occur (handled via PUT /api/profile), the backend uses a Spring Application Event Multicaster (@EventListener) to process notifications asynchronously without blocking the main thread.
3. **Database Notification Queue:** An event handler inserts a record into the PostgreSQL notification_queue table:
INSERT INTO notification_queue (customer_id, title, body, status, created_at)
VALUES (?, 'Security Profile Update', 'Your profile details were updated.', 'PENDING',
NOW());
4. **Client Queue Hydration:** The /notifications view calls the /api/notifications endpoint, fetching pending alerts from the database and updating the UI state dynamically.

Flow H: AI Financial Assistant Real-time Widget

- **Target Routes:** /chatbot
- **API Interconnects:** POST /api/chatbot

Technical Breakdown:

1. **Interactive Interface:** An interactive chat UI is exposed as a floating launcher or a dedicated /chatbot page.
2. **Secure Context Gathering:** When a user queries the AI assistant (e.g., "What was my highest transaction this month?"), the backend interceptor extracts the authenticated user session context before routing the prompt to the AI model.
3. **Database Context Injection:** The service executes a safe parameterized database query to fetch current user data, then constructs an augmented LLM prompt containing both user query context and system rules.
4. **Streaming Response Layout:** The response payload is streamed back to the client UI, keeping the interface responsive during complex operations.

3. Core Database Schema Design (PostgreSQL)

Below is the database schema design optimized for transaction tracking, indexing, and foreign key integrity constraints.

```
-- Disable automatic transaction commits to ensure clean schema initialization
BEGIN;
```

```
-- 1. CUSTOMERS TABLE
```

```
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  nic_number VARCHAR(20) UNIQUE NOT NULL,
  avatar_id VARCHAR(50),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. BANK ACCOUNTS TABLE
```

```
CREATE TABLE bank_accounts (
  account_number VARCHAR(20) PRIMARY KEY,
  customer_id UUID NOT NULL REFERENCES customers(id) ON DELETE RESTRICT,
  account_type VARCHAR(20) NOT NULL CHECK (account_type IN ('CHECKING', 'SAVINGS',
'LOAN')),
  balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00 CHECK (balance >= 0.00),
  currency VARCHAR(3) DEFAULT 'USD',
  status VARCHAR(20) DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'SUSPENDED', 'CLOSED')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 3. TRANSACTIONS TABLE
```

```
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sender_account VARCHAR(20) REFERENCES bank_accounts(account_number),
  recipient_account VARCHAR(20) REFERENCES bank_accounts(account_number),
  amount DECIMAL(15, 2) NOT NULL CHECK (amount > 0.00),
  category VARCHAR(50) NOT NULL,
```



```

description TEXT,
transaction_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. SAVINGS GOALS TABLE
CREATE TABLE savings_goals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id) ON DELETE CASCADE,
  goal_name VARCHAR(100) NOT NULL,
  target_amount DECIMAL(15, 2) NOT NULL CHECK (target_amount > 0.00),
  saved_amount DECIMAL(15, 2) NOT NULL DEFAULT 0.00 CHECK (saved_amount <=
target_amount),
  target_date DATE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. NOTIFICATION QUEUE
CREATE TABLE notification_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id) ON DELETE CASCADE,
  title VARCHAR(150) NOT NULL,
  body TEXT NOT NULL,
  status VARCHAR(20) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'SENT', 'FAILED')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- PERFORMANCE TUNING INDEXES
CREATE INDEX idx_accounts_customer ON bank_accounts(customer_id);
CREATE INDEX idx_transactions_sender ON transactions(sender_account);
CREATE INDEX idx_transactions_recipient ON transactions(recipient_account);
CREATE INDEX idx_transactions_date ON transactions(transaction_date);
CREATE INDEX idx_notifications_status ON notification_queue(customer_id, status);

COMMIT;

```

4. End-to-End Technical Routing Matrix

This matrix maps each Next.js user path to its corresponding Spring Boot REST controller mapping and targeted database table queries.

Next.js	Controller	HTTP Verb	System Layer	Primary
---------	------------	-----------	--------------	---------

Frontend Path	Endpoint Mapping			PostgreSQL Query Operations
/	<i>None (Static Asset)</i>	GET	Static Landing	None (Direct static routing assets)
/login	/api/auth/login	POST	Auth Subsystem	SELECT * FROM customers WHERE username = ?;
/sign-up	/api/auth/signup	POST	Auth Subsystem	INSERT INTO customers (id, ...) VALUES (...);
/dashboard	/api/accounts	GET	Account Engine	SELECT * FROM bank_accounts WHERE customer_id = ?;
/bank-transfer	/api/transfer	POST	Ledger Processing	Exclusive locks and atomic transaction balance updates
/pay-bills	/api/bill-payments	POST	Payment Subsystem	Updates user balances and inserts transaction records
/smart-spend	/api/smart-spend	GET	Analytics Engine	Analytical aggregation grouped by category
/savings-goals	/api/savings-go	GET/POST/PUT	Wealth	Full CRUD

	als		Management	operations against the savings_goals table
/e-statement	/api/statement	GET	Audit System	Streaming data selections across target date ranges
/settings	/api/profile	PUT	Config Manager	UPDATE customers SET ... WHERE id = ?;
/notifications	/api/notifications	GET	Event Interceptor	SELECT * FROM notification_queue WHERE customer_id = ?;
/chatbot	/api/chatbot	POST	Intelligent Agent	Secure context retrieval using read queries

5. Development Log, Verification & Remediation Actions

This section details development anomalies encountered during the transition from mock datasets to a strict Java and PostgreSQL environment, along with their respective resolutions.

1. Concurrent Transaction Conflicts (Race Conditions)

- **Issue:** Concurrent automated testing revealed that rapidly firing duplicate transfer API requests allowed users to spend past their actual balances, creating negative balances (race conditions).
- **Root Cause:** The default query transaction isolation level was set to READ_COMMITTED, which did not prevent concurrent threads from performing balance evaluations simultaneously before writing balance updates.
- **Remediation:** Configured the database query method to use explicit locking (FOR UPDATE) on balance checks and upgraded the transactional context boundary to a

serializable isolation level (Isolation.SERIALIZABLE).

2. Node.js Worker Compilations Errors (spawn EPERM)

- **Issue:** The compilation sequence failed on build pipelines, throwing system-level spawn EPERM terminal exceptions.
- **Root Cause:** Highly restrictive OS sandbox parameters blocked child build processes spawned during clean project compilation sweeps.
- **Remediation:** Executed the Next.js production build command with explicit elevated permissions (run-as administrator or targeted sudo exemptions) to enable successful production compilation.

3. File Character Corruption Errors (Mojibake)

- **Issue:** Certain system configurations displayed corrupted Unicode values or unparseable characters.
- **Root Cause:** Local scripting configurations wrote non-standard encoding streams, causing compilation issues with some external files.
- **Remediation:** Normalized all system project files to standard UTF-8 without a BOM signature and replaced any non-ASCII characters with safe fallback values.

6. Strategic Implementation Roadmap

Phase 1: Database Migration (Postgres Core Setup)

└─ Phase 2: Core Spring Boot Service Implementation

└─ Phase 3: Middleware Authentication & JWT Handshake Integration

└─ Phase 4: Full UI/UX Verification & Headless PDF Generation

1. **Phase 1: Database Migration & Deployment:** Deploy the standard PostgreSQL schema structure, initialize database indices, and establish clean connection pooling boundaries.
2. **Phase 2: Core Business Logic Implementation:** Set up the main Spring Boot Java modules, establish Hibernate mappings, and wrap critical financial ledger write operations inside transactional boundaries.
3. **Phase 3: Integration and Client Authentication:** Wire the custom middleware security layer (proxy.ts) to intercept API requests, unpack secure JWT browser cookies, and validate claims against active backend authentication sessions.
4. **Phase 4: Global Verification and Artifact Compilation:** Run testing sweeps to confirm project health, correct any layout issues, and print clean system reports directly into PDF artifacts using headless printing tools.